

lab 2

Lab setup

Before starting the OpenMP exercises, we prepared the environment and loaded the MNIST dataset.

First, we set up a Linux (Ubuntu/WSL) environment to ensure proper support for OpenMP and required system headers. Then we installed the necessary C++ build tools (`g++` , `build-essential`) to compile our programs.

Next, we downloaded the MNIST dataset files from the CVDF mirror:

- `train-images-idx3-ubyte.gz`
- `train-labels-idx1-ubyte.gz`

We unzipped these files to obtain the raw binary `.ubyte` dataset files.

After that, we created the header file `mnist_loader.h` , which:

- Defines a `MnistEntry` structure (784 pixels + 1 label)
- Reads the MNIST binary file headers
- Converts big-endian integers to host format
- Loads all images and labels into a `std::vector<MnistEntry>`

Finally, we created a small `main.cpp` program to test the loader and verified that:

- The dataset loads correctly
- 60,000 images are read
- Labels are accessible

At this point, the dataset is successfully loaded and ready to be used in the OpenMP exercises.

```
gunzip train_labels_10x1ubyte.gz
root@Youa-Laptop:~/mnist_lab# ./mnist_program
Loading 60000 images...
Total images loaded: 60000
First label: 5
root@Youa-Laptop:~/mnist_lab# |
```

Exercise 1

The idea is:

In a real ML pipeline, we often have **independent tasks** like:

- Loading configuration file
- Loading model weights

These tasks do NOT depend on each other.

So instead of running them one after the other (sequentially), we can run them **at the same time** using OpenMP.

This is called:

Task Parallelism

```
// Note: The total time should be
{
    load_config();
    load_weights();
}
```

```
#pragma omp parallel sections
{
    #pragma omp section
    load_config();

    #pragma omp section
    load_weights();
}
```

```
./ex1
Thread Thread 5: Loading Model Config...
4: Loading Pre-trained Weights...
Weights Loaded.
Config Loaded.
Loading MNIST dataset...
Loading 60000 images...
Setup Time: 2.10551s
root@Youa-Laptop:~/mnist_lab# nano ex1.cpp
```

I executed two functions in parallel using OpenMP `parallel sections`.

Because each function sleeps for 2 seconds, running them in parallel reduces the total execution time from about 4 seconds to about 2 seconds.

The different thread numbers shown in the output confirm that the tasks ran concurrently on separate threads.

Exercise 2

We will now simulate a grid search to find the hyperparameter combination that minimizes the loss.

The nested loops test all (epoch, batch) pairs and compute a loss value for each combination.

When parallelizing the loops, multiple threads may try to update `min_loss` at the same time, causing a race condition.

so I will use `reduction(min:min_loss)` so we ensure each thread keeps a private minimum and safely combines them at the end.

```
for (int i = 0; i < epochs; i++) {
    for (int j = 0; j < batches; j++) {
        // A convex function with minimum at (50, 50)
        double x = i - 50;
        double y = j - 50;
        double current_loss = (x * x) + (y * y) + 0.5;
    }
}
```

```
// Parallelize nested loops using OpenMP reduction
#pragma omp parallel for collapse(2) reduction(min:min_loss)
for (int i = 0; i < epochs; i++) {
    for (int j = 0; j < batches; j++) {
        // A convex function with minimum at (50, 50)
        double x = i - 50;
        double y = j - 50;
        double current_loss = (x * x) + (y * y) + 0.5;
    }
}
```

We use `collapse(2)` to merge the two nested loops into one large loop so

OpenMP can distribute the work more evenly across threads. Without

`collapse(2)` .. Only the **outer loop (i)** is divided among threads.

The condition to use collapse is to have a perfect nested loop means now code in between

```
root@Youa-Laptop:~/mnist_lab# ./ex2
Best Hyperparameter Loss: 0.5
root@Youa-Laptop:~/mnist_lab# |
```

Exercise 3

In this exercise we simulate the **evaluation phase** of a machine learning model.

We:

1. Go through the MNIST dataset
2. Simulate a prediction (random guess 0–9)
3. Compare prediction with the true label
4. Count how many predictions are correct

At the end, we compute: $\text{Accuracy} = \text{total_correct} / \text{total_processed}$

If the prediction matches the real label, we increase `total_correct` and we always increase `total_processed`

If we parallelize the loop, multiple threads may try to update these variables at the same time, causing a race condition and wrong counts.

We solved this by using

```
reduction(+:total_correct, total_processed)
```

so each thread counts separately and OpenMP safely adds the results together at the end.

```

root@Youa-Laptop:~/mnist_lab# ./ex3
Loading 60000 images...
Accuracy: 9.85%
Evaluation Time: 0.00960631 seconds
root@Youa-Laptop:~/mnist_lab# nano ex3.cpp
root@Youa-Laptop:~/mnist_lab# nano ex3.cpp
root@Youa-Laptop:~/mnist_lab# g++ -O2 -fopenmp ex3.cpp -o ex3
root@Youa-Laptop:~/mnist_lab# ./ex3
Loading 60000 images...
Accuracy: 9.915%
Evaluation Time: 0.00139874 seconds

```

We noticed that, unexpectedly, the serial code took less time to compute compared to the parallel version. This happens because creating and managing multiple threads introduces overhead, such as splitting the loop, scheduling work, and combining results at the end using reduction. Since the work inside each iteration is very small, the thread overhead becomes larger than the performance gain. As a result, the serial version runs faster because it avoids these parallel management costs.

to optimize pytorch we use couda and multicore program??

Exercise 4